

FOR FAMILY EYES — TECHNICAL REVIEW

A Technical Conversation

PadelMind — the full three-phase build plan, architecture, contractor situation, and seven technical questions where your opinion would change the call.

PREPARED FOR

Arjun Maheshwari

READING TIME

20–30 minutes

REQUIRED RESPONSE

Seven technical questions — \$7

AUTHORED

Manoj Maheshwari · 2026-07-09

A conversation summary and a request for your opinion

Why I am sharing this: A lot has changed since the first documents I sent you. The plan has been stress-tested against a solo-founder reality, a contractor engagement is in motion, and the architecture has been scoped in significant detail. Before we go further, I want your read on whether the technical decisions we have made hold up — and whether the sequencing makes sense to you. You know this domain better than I do on the tech side. I want your honest opinion, not your reassurance.

1. What Changed Since the Last Documents

The original plan assumed a founding team of three from day one — CEO (me), ML co-founder (target: João Silva), and a coach CCO. That team has not materialised. The cost constraint is real: I am currently the only person committed.

That forced a complete rethink. The question became: **what can one person build that is still a genuine “wow” product, without a coach, without an ML co-founder, and without the shot classifier that was supposed to be the moat?**

The answer that emerged: **the shot classifier is not the first wow. The first wow is this.**

A padel player finishes their match at 7:30pm. At 7:42pm, their WhatsApp receives three messages: a text summary of the match, a court heatmap showing where they spent each rally, and a 60-second highlight reel of their top five rallies — auto-clipped and delivered.

No app to download. No login. The club sends one WhatsApp command. Everything else is automatic.

This product does not need a shot classifier. It does not need pose estimation. It does not need ball tracking. It needs player position tracking, a heatmap renderer, a rally detector, and an ffmpeg clipper. Four things I can mostly build with AI tooling on the infrastructure side. The CV pipeline is the only piece that requires a specialist — and that is where a contractor named Sasank comes in.

2. The Three-Phase Build Plan

Phase 0 — What I Build Alone (Now, 12 weeks)

This is everything on the infrastructure and delivery side that sits within my existing skillset and tools.

WHAT	STACK	STATUS
Supabase database schema	Postgres — same as AX	Ready to build
Cloudflare R2 video storage	R2 + Workers	Ready to build
Job queue (Pi upload → GPU job)	Cloudflare Worker + Upstash Redis	Ready to build
WhatsApp delivery	AX Meta Cloud API router (already running)	Wire-up only
Player PWA (match history, heatmaps, highlights)	React + Supabase + Cloudflare Pages	Ready to build
Club WA command parser (<code>PADEL START / END</code>)	AX router extension	Wire-up only

The WhatsApp delivery infrastructure is already running in production on AutomationXpert.

Sending a match report to a player's WhatsApp is a 2–3 hour integration, not new infrastructure.

What I cannot build alone: The CV pipeline — everything that happens between a video arriving in R2 and the heatmap + highlight being produced. That is Sasank's scope.

Phase 1 — What the Contractor (Sasank) Builds (12 weeks, paid engagement)

Sasank is a developer in Hyderabad who has done prior work on ball tracking in an assignment project. He has not done player detection or pose identification before — which is relevant and I will come back to it.

His scope has six components:

Component 1 — Raspberry Pi Edge Software The Pi (Model 5, 8GB, 1TB NVMe) sits at the club in a weatherproof enclosure. It receives the camera RTSP stream, records 10-minute MP4 chunks, detects when the match ends, concatenates the full match video, and uploads it to R2. The trigger is a WhatsApp command from the club. Motion-based end-detection is the fallback.

Component 2 — GPU Inference Worker (RunPod) A Docker container on a RunPod RTX 4000 Ada (20GB VRAM) that downloads the match, runs YOLOv8m + ByteTrack for player detection and tracking, computes court positions via homography from 12 pre-clicked keypoints, and produces a per-frame positions JSON for all four players.

Base library: `padel_analytics` (MIT licence, open-source, authored by João Silva). The repo already has YOLOv8 + ByteTrack + court homography implemented — Sasank strips the Streamlit UI and adapts it to run headlessly inside Docker on RunPod.

Component 3 — Heatmap Renderer From the positions JSON (player coordinates in court metres), renders one heatmap PNG per player using matplotlib + scipy gaussian smoothing. Dark background, fire colourmap, court lines overlaid. Delivered to WhatsApp as an image.

Component 4 — Rally Detector Purely algorithmic — no ML. Computes player velocity from consecutive positions. A rally is any continuous window where ≥ 2 players are present and moving above a threshold. Produces the top-8 rally windows by duration. These feed the highlight clipper.

Component 5 — ffmpeg Highlight Clipper Clips the top 5 rally windows from the match video, adds 1-second padding, stitches into a single 60–90 second 9:16 MP4 under 15MB. Delivered to WhatsApp as a video.

Component 6 — Court Calibration Tool A single-page browser tool used once at camera install. The installer uploads a still frame from the camera and clicks 12 court keypoints. The tool saves the pixel coordinates to Supabase. These coordinates are included in every GPU job so the homography is computed at runtime without re-clicking.

Phase 2 — The Moat Layer (After Phase 1 is live, team-dependent)

Phase 2 requires two things Phase 1 does not have: a **coach CCO** (to write the shot taxonomy rubric and label training data) and an **ML specialist** for the shot classifier.

Sasank’s prior ball tracking experience is directly relevant here — this is where he becomes most valuable.

WHAT	DEPENDENCY	STATUS
Ball tracking (TrackNet retrained on padel)	Sasank — his prior domain	Phase 2 Sasank scope
Pose identification (YOLOv8-pose, 17 keypoints)	GPU worker extension	Phase 2 Sasank scope
Shot taxonomy rubric (14 shot classes, quality 1–5)	Coach CCO	Blocked on coach hire
Data labelling pipeline (Streamlit tool + labellers)	Coach + 2 labellers	Blocked on coach hire
Shot classifier (custom MLP on ~390-dim feature vector)	ML specialist	Blocked on ML hire
Skill score engine	Classifier output	Blocked on classifier
Drill recommendations	Coach rubric + classifier	Blocked on coach hire

Phase 2 is the actual moat. Phase 1 is the demo that recruits the coach and the ML person — and potentially attracts the first angel.

3. The Technical Architecture — Full Pipeline

```
[IP Camera – Reolink RLC-823A 4K PoE]
| RTSP H.265 stream (local network)
▼
[Raspberry Pi 5 – 8GB RAM, 1TB NVMe, 4G/5G modem]
MediaMTX: RTSP relay + 10-min MP4 segments
match_daemon.py: chunk concat + R2 upload on match end
| boto3 upload (~3.5 GB per 90-min match)
▼
[Cloudflare R2 – video storage, zero egress fees]
| upload-complete webhook
▼
[Cloudflare Worker + Upstash Redis queue]
| job dispatch
▼
[RunPod RTX 4000 Ada – serverless, pay-per-second]
padel_analytics fork (headless):
| — Court homography (from stored 12-point keypoints)
| — YOLOv8m player detection
| — ByteTrack player ID tracking
| — Position projection → court metres (x, y) per frame
| — Rally detector (velocity-based, no ML)
| — Heatmap renderer (matplotlib, 4 PNGs)
| — ffmpeg highlight clipper (top-5 rallies, <15MB MP4)
| outputs: positions JSON + 4 heatmap PNGs + highlight MP4 → R2
| callback webhook to Cloudflare Worker
▼
[Supabase – match metadata, player identities, output URLs]
▼
[AX Meta Cloud API – WhatsApp delivery]
3 messages to all 4 players:
| — Text summary (match duration, rally count, court zones)
| — Heatmap PNG per player
| — Highlight MP4 (shared, same for all 4)
▼
[Player PWA – React, Cloudflare Pages, Supabase auth]
Full match history, all heatmaps, all highlights
Auth: phone OTP (no password, no app store)
Link in every WA message opens the player's own PWA profile
```

End-to-end target: match ends → WhatsApp message arrives in under 15 minutes.

4. Hardware — Single Camera, Phase 1 (Plan A)

One camera per court. Deliberately simple for the pilot.

ITEM	SPEC	INR
Camera	Reolink RLC-823A 4K PoE IP66	₹12,000–18,000
PoE Switch	TP-Link TL-SG1005P	₹3,000
Raspberry Pi 5	8GB + active cooler	₹8,500
NVMe SSD	1TB WD Green + Pi 5 NVMe HAT	₹5,500
4G/5G modem	Jio/Airtel USB + unlimited SIM	₹3,000 + ₹1,000/mo
Enclosure + cabling + mount	IP65 box, Cat6, bracket	₹3,800
Total per court		₹35,800–41,800

Running cost per court per month (at 120 matches): GPU inference on RunPod RTX 4000 Ada serverless: ₹1,464 R2 storage + data SIM: ₹1,150 Total COGS: ~₹2,614

At ₹5,000/court/month subscription: ₹2,386 net margin. 48% gross margin at this pricing tier.

Dual-camera (two Pis, position-merge across both views) is Phase 2. The accuracy gain is real but the complexity cost is 4–5 weeks additional build time and ₹35k additional hardware. Not worth it before the first paying club.

5. The Contractor Situation — Sasank

Profile: Developer in Hyderabad. Has done ball tracking work in a prior assignment project. Has not done player detection (YOLOv8) or pose identification previously.

The irony in the sequencing: The thing he knows (ball tracking) is Phase 2. The thing Phase 1 needs most (player detection + tracking) is his gap.

Why we are proceeding anyway: 1. His ball tracking background transfers the surrounding infrastructure — GPU inference pipelines, Docker/CUDA, video processing with ffmpeg, RunPod-style workers. He is not starting from zero. 2. The `pade1_analytics` OSS repo already has YOLOv8 + ByteTrack implemented and working. His job is to strip the UI and containerise it — adapting existing code, not writing a detection model from scratch. 3. Phase 2 is where he becomes most valuable. Keeping him through Phase 1 means he comes into Phase 2 with the full pipeline in his head.

The engagement model: - He has asked for upfront payment commitment before starting — reasonable for a freelancer - Our approach: get his effort estimate first, then structure milestone payments tied to verifiable acceptance tests - Each milestone payment releases only when we run the acceptance test ourselves — not when he declares it done - 20–25% on contract signing (commitment signal), remaining 75–80% split across component milestones - Estimated total cost: ₹90,000 – ₹1,40,000 for Phase 1 depending on his estimate

We have not asked him for a proof of concept without payment. He declined that framing. The milestone structure above is the alternative — he proves each component works before receiving payment for it.

6. GPU Infrastructure

Phase 1: RunPod serverless, RTX 4000 Ada (20GB VRAM). Pay-per-second, no idle cost. ~₹12/match. No India data centre but acceptable at Phase 1 volume.

Phase 2 (25 courts): AWS ap-south-1 (Mumbai) spot instances — G5.xlarge (A10G, 24GB). Mumbai DC eliminates video upload latency to EU. ~₹5/match at sustained load, 4× cheaper than RunPod at scale.

Phase 3 (200+ courts): Edge inference at club level (Jetson AGX Orin, ₹80k hardware per club). Eliminates cloud GPU cost entirely — only outputs (JSON + clips) travel to cloud.

7. Questions I Need Your Technical Opinion On

These are the decisions where I have made a call but am not confident it is the right one. I want your honest read.

Q1 — YOLOv8m at 720p, frame-stride 6: is this accurate enough for heatmaps? The plan is to downsample 4K footage to 720p before inference, and process every 6th frame (5 fps effective) for player tracking. This cuts inference time by ~4× compared to 1080p at every frame. The heatmap only needs approximate court positions — it is not a biomechanics analysis. But if 5 fps tracking produces too many ID switches or gaps, the heatmap zones will be wrong. What is your read on whether this is sufficient accuracy for the use case?

Q2 — Is the `padel_analytics` repo a reliable base, or does it have architectural issues we should know about? The repo (github.com/Joao-M-Silva/padel_analytics) is MIT-licensed, ~2,000 lines, Streamlit-first, uses YOLOv8 + ByteTrack + court homography. We are forking it, stripping the UI, and running it headlessly in Docker. I have read the code. It looks well-structured. But I am not a CV engineer. Is there anything about this class of repo (Streamlit-wrapped research code) that creates headaches when you try to productionise it?

Q3 — Is the rally detector reliable without ball tracking? The rally detector uses player velocity from position data to determine whether a rally is in progress. It does not use ball trajectory — ball tracking (TrackNet) is Phase 2. The concern: a player warming up alone at the baseline might register as a "rally" if they are moving. Is player velocity alone a reliable-enough signal, or do you think the false positive rate will be too high to produce a usable highlight reel?

Q4 — RTX 4000 Ada 20GB vs RTX 3090 24GB for Phase 1? We picked RTX 4000 Ada for cost (₹1.11/min vs ₹1.82/min on RunPod). The 20GB VRAM handles YOLOv8m comfortably in Phase 1. But in Phase 2 when we add YOLOv8-pose + TrackNet running sequentially, VRAM headroom tightens to ~8–10GB free. Is that enough headroom, or should we start on RTX 3090 24GB to avoid a GPU migration mid-project?

Q5 — Is Sasank's background sufficient for Component 2? Component 2 (the GPU inference worker) is the hardest part of Phase 1 — adapting YOLOv8 + ByteTrack to run headlessly inside Docker on RunPod, and hitting a 90-min match processed in under 6 minutes on an RTX 4000 Ada. Sasank has GPU inference experience from ball tracking work but has not used YOLOv8 or ByteTrack before. The `padel_analytics` repo does the heavy lifting — his job is containerisation and adaptation. Is this a realistic ask for someone with his background, or is Component 2 likely to be the failure point?

Q6 — Phase 2 sequencing: ball tracking before or after shot classifier? Phase 2 adds three things: ball tracking (Sasank's strength), pose estimation (YOLOv8-pose), and a shot classifier (needs a coach rubric + ML specialist). My instinct is to do ball tracking first (Sasank can start without the coach) and shot classifier later (coach-dependent). Is that the right order, or does the shot classifier depend on ball trajectory data from TrackNet in a way that makes them simultaneous dependencies?

Q7 — Single camera at back wall vs centre-mount: does it matter for Phase 1? We locked one camera per court mounted 4–5m high behind one back glass wall. This means players close to the camera-side baseline are partially occluded by the glass structure. The alternative is a centre-mount at the net post (better symmetry, less occlusion) but structurally harder to install. For heatmap accuracy specifically — does back-wall mounting produce a material blind spot that would make the heatmap misleading?

8. What I Am Asking You to Do

Read this document. Tell me where the plan is solid and where it has a problem you can see that I have missed.

If you think any of the seven questions above has an obvious answer from your vantage point, tell me. If you think the whole Phase 1 architecture is wrong — that there is a better way to get from a padel court to a WhatsApp message in 15 minutes — tell me that too. The plan is not locked. It is the best thinking I have with the information I have.

The one thing I am not asking: do not tell me it looks fine if you can see a gap. The cost of a wrong technical decision now is much higher than the cost of a hard conversation today.

Prepared by Manoj Maheshwari · PadelMind by AutomationXpert · 2026-07-09 Single source of truth:

`~/Documents/padel-clone/` — *this document is a snapshot for review. Related documents:*

`PADEL_AI_VENTURE_PLAN.md` (strategy) · `SASANK_SOW_PHASE1.md` (contractor brief) ·

`INTERNAL_EFFORT_REFERENCE.md` (internal benchmarks)

Companion document set: Strategic Venture Plan · Project Report · Technical Feasibility · Commercial Report · Base Club Offer · Technical Review Request.

Single source of truth maintained in `~/Documents/padel-clone/`.